OpenGL unit 2 summary

OpenGL — Unit 2 (Explained clearly, with a balanced mix of paragraphs and a few short lists)

This unit introduces why OpenGL exists, how it behaves as a state machine, the kinds of things you do with it, and some example programs and concepts (Sierpinski gasket, hidden-surface removal). I'll walk you through the ideas in plain technical language so the concepts stick — mostly in paragraphs, with short lists where a compact summary helps.

Why OpenGL? What is it, really?

OpenGL is not a programming language; think of it instead as a portable graphics *runtime library* — a standardized interface that programs call to produce 2D and 3D graphics. It gives you a set of functions that map to capabilities of graphics hardware. Because OpenGL is device- and platform-independent, code you write with OpenGL can run across different operating systems and GPUs (subject to driver support). The API is the contract: it specifies what calls you can make and how they behave; the driver implements that contract for each piece of hardware.

The big advantage is that OpenGL exposes powerful, highly optimized algorithms (originally developed/packaged by SGI) so you can generate sophisticated visuals without implementing low-level rasterization, clipping, or transformation math yourself. In short: you focus on *what* to draw and *how* it should look; OpenGL, together with the GPU/driver, focuses on *how* to render that quickly and correctly.

OpenGL as a state machine

A useful mental model for OpenGL is a state machine: there's a global set of states (current color, current transformation matrices, enabled/disabled features, texture bindings, shader programs, etc.). When you call OpenGL functions, you change or query those states; subsequent draw calls use whatever the current state is.

Because of this, small mistakes like forgetting to reset a color or forgetting to change the matrix mode can have visible effects on later draw calls. It also means that to understand what will be drawn you must think of the last set of state changes that affect the drawing pipeline. This stateful design historically simplified GPU drivers and allowed batching of commands for performance, although it also demands careful bookkeeping in your code.

The "black box" model of a graphics package

When the slides call a graphics system a "black box," they mean you treat the rendering engine as something you feed inputs into and get rendered pixels out of — you don't need to know the inner hardware details. Your program issues function calls (e.g., to draw primitives, set colors, change matrices), those become the input; the screen or framebuffer is the output. The internal pipeline (vertex

processing, clipping, rasterization, fragment processing) is abstracted away, but the API gives you hooks to configure it.

This abstraction is useful because it separates concerns: you write geometry, materials, camera setup; the driver/GPU handles scheduling, memory management, and efficient rasterization.

What kinds of graphics functions exist?

OpenGL's functions fall into familiar conceptual groups. Instead of just listing them, here's what each group means and why it matters:

1. **Primitive functions** let you tell OpenGL the raw geometric objects you want to draw: points, lines, triangles, polygons, and text. In older (immediate-mode) OpenGL you'd bracket vertex definitions between **`glBegin()`** and **`glEnd()`** and specify each vertex with **`glVertex*()`** calls. For example:

   ```
   c


   glBegin(GL_LINE_LOOP);
     glVertex3f(-0.3f, -0.3f, 0.0f);
     glVertex3f( 0.0f,  0.3f, 0.0f);
     glVertex3f( 0.3f, -0.3f, 0.0f);
   glEnd();
   ```

   That draws a triangular outline. Modern OpenGL replaces this immediate-style with vertex buffers and vertex array objects for performance, but the concept — assembling primitives from vertices — is the same.

2. **Attribute functions** control how primitives look. Attributes include color, material parameters, line width, polygon fill mode, and clear color for the framebuffer. They're the cosmetic or rendering-style knobs you turn. Example calls are **`glColor3f(1.0, 0.0, 0.0)`** to set the current drawing color to red, or **`glClearColor(1.0, 1.0, 1.0, 1.0)`** to set what color the window clears to. Attributes are part of the OpenGL state machine: once set, they affect subsequent drawing until changed.

3. **Viewing functions** define the camera and the region of space you want mapped into the final image. These include setting the viewport (**`glViewport`**), selecting projection vs. modelview matrix modes (**`glMatrixMode(GL_PROJECTION)`** vs **`glMatrixMode(GL_MODELVIEW)`**), and functions to define orthographic or perspective projections (**`gluOrtho2D`**, **`gluPerspective`**, or manually loading projection matrices). Viewing functions let you create front/top/side views, zoom in/out, and define the coordinate system used for drawing.

4. **Transformation functions** are how you move and orient geometry. With **`glTranslatef()`**, **`glRotatef()`**, and **`glScalef()`** you transform the current matrix so that subsequent vertex coordinates are transformed accordingly. Model transformations position objects in the world; view transformations position the camera; projection transformations control how depth and perspective are mapped to screen space. In modern OpenGL you manipulate matrices on the CPU and upload them to shaders rather than relying on the fixed-function matrix stack, but the conceptual roles haven't changed.

5. **Input functions** allow your program to react to user actions: mouse movement, mouse clicks, keyboard presses, tablet events. In GLUT, for instance, you register callbacks like **`glutMouseFunc(mouse)`** to get mouse events. Interactive applications rely on these callbacks to update scene state (e.g., rotate a model when the user drags the mouse).

6. **Control functions** are how your app initializes and manages the windowing system integration: creating windows, selecting display modes, setting initial window size and position, and controlling the render loop. Examples are `glutInitDisplayMode()`, `glutInitWindowSize()`, and `glutInitWindowPosition()`. These are the bridge between your program and the OS windowing framework.
7. **Query functions** let you ask OpenGL about its current state or capabilities: what the current boolean flags are, integer parameters, the contents of framebuffers, or device capabilities. Calls like `glGetBooleanv()` and `glGetIntegerv()` are used to read back state for debugging, conditional logic, or adaptive behavior depending on hardware limits.

Primitive vs attributes vs viewing — how they interact in practice

It helps to picture a simple rendering sequence: you set some attributes (color, polygon mode), set your viewing/projection matrix so the camera sees a certain part of the world, and then issue primitive draw calls (triangles) that are transformed by the current modelview/projection matrices and rendered with the current attributes. Input callbacks can change matrices or attributes in response to user events. Queries can check whether certain features are enabled or what the device limits are, and control functions bootstrap everything.

Because many of these are global states, you should be deliberate: set only what's necessary and restore state when you're done if other parts of your program rely on previous settings (or use local matrix stacks or encapsulate state changes).

Two-dimensional viewing as a special case of 3D viewing

2D viewing is simply 3D viewing with depth either ignored or flattened. By using an orthographic projection (e.g., `gluOrtho2D()` or `glOrtho()`), coordinates map linearly to screen space without perspective distortion, which is exactly what you want for many 2D applications (GUIs, simple plots, 2D games). The same transformation pipeline applies: vertices → model/view transform → projection → viewport mapping. You simply choose projection and transformations that make the Z dimension irrelevant or constant.

Thinking of 2D as a subset of 3D is powerful: you can reuse 3D tools (transformations, lighting, depth testing) in a 2D project when needed.

Interaction with the window system

OpenGL itself doesn't create windows; it relies on a window-system-specific layer or a utility library (GLUT, GLFW, SDL, platform APIs) to create a window and an OpenGL context. That context is what binds the OpenGL state to a drawable surface. Control functions in your chosen windowing toolkit set display modes (double buffering, depth buffer bits, stencil bits), window size/position, and register callbacks. The toolkit also provides the event loop and dispatches input events to your callbacks. So interaction with the OS window system is essential: the OpenGL API handles rendering once the context exists, but windowing code handles user interaction and the native window.

Sierpinski Gasket programs — triangles and 3D

The Sierpinski gasket (triangle) is a classic fractal and a great exercise for learning recursive drawing and basic OpenGL primitives. The 2D version is generated by repeatedly subdividing a triangle into smaller triangles and removing the central one — visually, it's an infinite recursion pattern of triangular holes.

A typical simple implementation for learning OpenGL draws the initial large triangle and recursively draws smaller triangles by computing midpoints of edges and issuing vertex calls for the three sub-triangles until some recursion depth is reached. This practice helps you understand:

- Primitive drawing (triangles),
- Recursive geometry generation,
- The effect of transformation and scaling.

Extending it to 3D makes the gasket a Sierpinski tetrahedron (or higher-dimensional analogues): you subdivide a tetrahedron into smaller tetrahedra. The 3D exercise introduces depth, perspective projection, and the need for hidden-surface removal (or ordered drawing) to show the 3D structure clearly. You'll also begin to use transformations to rotate the 3D gasket and observe the fractal from different viewpoints.

Hidden-surface removal (HSR)

Hidden-surface removal is the set of techniques used to determine which surfaces (or parts of surfaces) are visible from the camera and which are occluded by nearer geometry. This prevents the renderer from drawing fragments that should be hidden behind other geometry.

There are several approaches:

- **Depth (Z) buffer**: The most common modern approach. Each pixel stores the depth of the closest fragment drawn so far; when a new fragment is produced, its depth is compared to the stored value and the fragment is discarded if it is further away. OpenGL supports depth buffering and provides functions to enable and configure it (`glEnable(GL_DEPTH_TEST)` and setting up a proper depth buffer during window creation). The Z-buffer approach is hardware-accelerated and simple to use.
- **Painter's algorithm**: Draw from farthest to nearest so nearer objects overwrite farther ones. This can work for some scenes but fails with intersecting geometry and requires sorting.
- **Occlusion culling and more complex algorithms**: For performance on large scenes, techniques remove entire objects that are fully occluded before rasterization.

When you move from simple 2D shapes to 3D meshes (like the 3D Sierpinski gasket), enabling depth testing and properly clearing the depth buffer every frame are essential to get correct visuals.

Putting it all together — a small workflow example

Imagine you want to draw the 3D Sierpinski gasket and allow the user to rotate it with the mouse. The workflow is:

1. Initialize window and OpenGL context; request a depth buffer (control functions).
2. Set clear color and enable depth testing (attribute + control).
3. Set up projection (perspective) and initial modelview matrix (viewing + transformation).
4. Create a function that recursively builds the gasket and issues triangle primitives (primitive functions).
5. Register mouse and keyboard callbacks to adjust rotation angles (input functions).
6. In the display callback, clear the color and depth buffers, set up model transformations based on the user input, draw the gasket, and swap buffers.

This demonstrates how the different groups of functions cooperate: primitives describe geometry, attributes control appearance, viewing and transformations position the camera and objects, input allows interaction, and control handles the window/context lifecycle.

Quick reference summary (compact)

- OpenGL = portable graphics API (library), not a language. It provides hardware-accelerated rendering primitives and pipeline control.
- It behaves as a **state machine**: set states (colors, matrices, shaders), then draw.
- Function groups:
  - **Primitive**: draw geometry (**`glVertex*`**, **`glBegin`**/**`glEnd`** in old GL; now VBOs/VAOs).
  - **Attribute**: appearance (**`glColor*`**, **`glMaterial*`**, **`glClearColor`**).
  - **Viewing**: camera/projection (**`glViewport`**, **`gluOrtho2D`**, projection/modelview).
  - **Transformation**: move/rotate/scale (**`glTranslatef`**, **`glRotatef`**, **`glScalef`**).
  - **Input**: connect mouse/keyboard events (**`glutMouseFunc`**).
  - **Control**: window/context management (**`glutInitDisplayMode`**, **`glutInitWindowSize`**).
  - **Query**: read state/capabilities (**`glGet*`**).
- 2D is a special case of 3D (use orthographic projection).
- Hidden-surface removal is usually handled via the depth buffer in modern pipelines.
- Sierpinski gasket exercises help practice recursion, primitives, and 3D rendering concepts.

If you want, I can convert this into a one-page cheat-sheet you can print or a short set of annotated code examples (immediate-mode and modern VBO/shader-based), or prepare a short step-by-step lab to implement the 2D and 3D Sierpinski programs with depth testing and mouse interaction. Which would help you remember this material best?

The OpenGL Interface – Unit 2

Overview

The OpenGL interface consists of **three main libraries** that together form a complete environment for developing graphical applications. These libraries simplify the process of drawing geometric shapes, handling windows, and managing user input devices.

---

1. Graphics Library (GL)

The **Graphics Library (GL)** is the core part of OpenGL.
It provides functions that allow the creation of a small set of **basic primitives** such as points, lines, and polygons.

- All OpenGL functions begin with the prefix **`gl`**.
- These functions are stored in a library called **GL** (known as *OpenGL* in Windows).
- The GL library forms the foundation of all other OpenGL utilities.

---

2. Graphics Utility Library (GLU)

The **Graphics Utility Library (GLU)** builds upon the basic GL functions to provide higher-level functionality.

- It supports creation of **more complex primitives** such as spheres, cylinders, and curves.
- GLU functions are implemented using only GL functions internally, making it a true utility layer.
- These functions begin with the prefix **`glu`**.
- GLU is part of all standard OpenGL implementations, ensuring cross-platform compatibility.

Example:
`gluSphere()` – creates a sphere using underlying OpenGL primitives.

---

3. GL Utility Toolkit (GLUT)

The **GL Utility Toolkit (GLUT)** is used for managing the **window system and user input**.

- It allows OpenGL programs to run on different platforms without needing separate window libraries.
- Platform-specific equivalents include:
  - **GLX** – for X Window System (Linux/Unix)
  - **WGL** – for Windows
  - **AGL** – for Macintosh
- Instead of using different libraries, GLUT provides a **uniform interface** that handles:
  - Window creation
  - Keyboard and mouse input
  - Event handling

This toolkit offers the minimal functionality expected from any modern windowing system and is extremely useful for building cross-platform OpenGL applications.

Integration of the Libraries

In most systems, including all three headers is sufficient for using OpenGL:

```
c

#include <GL/glut.h>
```

This automatically includes **`gl.h`**, **`glu.h`**, and **`glut.h`**.
Your application program typically interacts with OpenGL through GLUT and doesn't directly reference the underlying platform libraries.

OpenGL Primitives and Attributes

OpenGL supports two major kinds of primitives:

1. **Geometric Primitives:**
   - Points, line segments, polygons, curves, and surfaces.
2. **Image (Raster) Primitives:**
   - Arrays of pixels used for textures or images.

These primitives are processed through the **geometric pipeline**, where operations such as transformation, projection, and clipping are applied before being sent to the **frame buffer**.

Defining Geometric Primitives

A geometric primitive is defined between the functions **`glBegin(type)`** and **`glEnd()`**, where **`type`** specifies how the vertices are interpreted.

Syntax:

```
c

glBegin(type);
glVertex*(...);
...
glEnd();
```

Common primitive types include:

- **`GL_POINTS`** – draws each vertex as a single pixel.
- **`GL_LINES`** – connects pairs of vertices with line segments.
- **`GL_LINE_STRIP`** – connects vertices in sequence, leaving the last unconnected.

- `GL_LINE_LOOP` – connects all vertices and closes the loop.

Programming 2D Applications

In OpenGL, both **2D and 3D points** are represented similarly.
A 3D point is a triplet `(x, y, z)` known as a **vertex** — a specific location in space. Vertices are the building blocks for all geometric objects.

- One vertex → Point
- Two vertices → Line segment
- Three vertices → Triangle or circle
- Four vertices → Quadrilateral

Vertices are defined using the `glVertex*()` function, where:

- `n` → number of dimensions (2, 3, or 4)
- `t` → data type (`int`, `float`, or `double`)
- `v` → specifies that arguments are passed as an array pointer

Example:

```
c


glBegin(GL_LINES);
glVertex3f(x1, y1, z1);
glVertex3f(x2, y2, z2);
glEnd();
```

Polygon Basics

Polygons are crucial in graphics because they form the basis for complex models.
A polygon can be rendered in multiple ways:

- Display only the **edges** (wireframe)
- Fill the **interior** with color or patterns
- Combine both edge and interior display

The efficiency of a graphics system is often measured by the **number of polygons rendered per second**.

Properties of a Polygon

To ensure that a polygon renders correctly, it must satisfy three properties:

1. **Simple:**
   Edges do not intersect. The interior is well-defined.

2. **Convex:**
   All interior angles are less than 180°. A line drawn through a convex polygon intersects its edges exactly twice.
   Examples: triangles, rectangles, trapezoids.
3. **Flat:**
   All vertices lie in the same geometric plane.
   In 3D graphics, this ensures proper lighting and shading calculations.

Polygon Types in OpenGL

OpenGL provides several polygon drawing modes:

- `GL_POLYGON` – forms a closed path and fills the interior.
- `GL_TRIANGLES` – each set of three vertices forms a triangle.
- `GL_QUADS` – each set of four vertices forms a quadrilateral.
- `GL_TRIANGLE_STRIP` – each new vertex, combined with the previous two, creates a new triangle.
- `GL_QUAD_STRIP` – two new vertices form a new quadrilateral with the previous two.
- `GL_TRIANGLE_FAN` – uses a central fixed point and creates connected triangles that share this point.

Curved Primitives and Tessellation

Curved shapes like **spheres**, **ovals**, or **parabolas** aren't directly defined as primitives in OpenGL.
They are created using a process called **tessellation**, which involves dividing a curved surface into a mesh of many small polygons.

For example:

- A **sphere** can be approximated by connecting many small triangles or quadrilaterals.
- The more subdivisions you create, the smoother the curved surface appears.

Summary

- **GL** → Basic primitive creation
- **GLU** → Higher-level primitives (uses GL internally)
- **GLUT** → Window system interface and input handling
- **Primitives** → Basic building blocks like points, lines, and polygons
- **Polygons** → Must be simple, convex, and flat
- **Tessellation** → Used to approximate curved surfaces

Together, these components make OpenGL a flexible, device-independent system for creating both 2D and 3D graphics applications efficiently.

Below is a focused, well-balanced explanation of the material you pasted — keeping closely to the content you provided, not adding unnecessary extras, and arranging it so it's easy to read and remember.

Approximating a Sphere

A sphere can't be drawn as a single primitive in classic OpenGL; instead you approximate it by tessellating the surface into many small polygons (quads or triangles). The usual approach is to treat the sphere as a grid of **latitude** and **longitude** lines and form strips (quad strips or triangle strips/fans) between successive latitude circles.

Mathematically, for a unit sphere (radius = 1) a point on the surface is parameterized by two angles (the slides use θ and Ø):

- $x(\theta, \varphi) = \sin\theta \cdot \cos\varphi$
- $y(\theta, \varphi) = \cos\theta \cdot \cos\varphi$
- $z(\theta, \varphi) = \sin\varphi$

(Here the slides call the second angle Ø or φ; think of φ as latitude — it moves from south to north — and θ as longitude around the equator.)

Practical details from the notes:

- If you fix φ and vary θ you trace a **circle of constant latitude** (parallel).
- If you fix θ and vary φ you trace a **circle of longitude** (meridian).
- To build the surface you generate points at fixed increments of θ and φ and connect them into quadrilaterals (or triangles).
- The recommended ranges: φ from −80° to +80° (stop short of the poles and handle poles specially), and θ from −180° to +180°.
- Convert degrees to radians before calling **`sin`**/**`cos`**: multiply degrees by $\pi/180$ (the slides use **`float c = 3.142/180;`**).

How the code layout works (high level, matching the slides):

1. Choose an angular step (e.g. 20°). The step controls smoothness: smaller step → smoother sphere, but more polygons.
2. For each latitude band (φ from −80 to +80 in increments of step) you compute φ radians and φ+step radians (two latitude rings).
3. Begin a **`GL_QUAD_STRIP`**. Then loop θ from −180 to +180, computing θ in radians for each step, and emit two vertices for each θ: one at (θ, φ) and one at (θ, φ+step). These paired vertices form the quad strip that covers the band between the two latitudes. Close the quad strip with **`glEnd()`**.
4. Near the top and bottom (the poles) use **`GL_TRIANGLE_FAN`** centered at the pole (0,0,±1). For each fan you set the pole vertex first, then emit the ring of vertices around the nearest latitude (φ = ±80° converted to radians). This cleanly stitches the pole area without degenerate quads.

This pattern — quad strips for the mid-latitude bands and triangle fans for each pole — is a standard, efficient way to tessellate a sphere in immediate-mode OpenGL.

Notes on quality/performance tradeoff: decreasing the angular increment (e.g., from 20° to 5°) raises polygon count and visual smoothness, while increasing CPU/GPU cost and memory for vertex buffers.

Text in OpenGL

OpenGL supports two broad ways to render text: **stroke** (vector) fonts and **raster** (bitmap) fonts. Each has a distinct behavior and trade-offs.

Stroke text

- Stroke fonts are described by line segments and curves (like any geometric object). Characters are built from vertices and line/curve primitives.
- Because they're geometric, stroke text behaves like other 3D objects: you can **translate, rotate, scale, light,** and view them from different camera angles. They can be filled or drawn as outlines.
- API example: **`void glutStrokeCharacter(void *font, int character)`** draws a stroke glyph.
- Advantages: flexible transformations, high-quality scaling (not blocky).
- Disadvantages: more memory and processing time (more vertex data and CPU work).

Raster text

- Raster fonts are bitmaps: each character is stored as a block of bits (a bit-block), and rendering is a fast bit-blit into the frame buffer.
- API example: **`void glutBitmapCharacter(void *font, int character)`.**
- Advantages: extremely fast to place into the framebuffer; simple to implement.
- Disadvantages: scaling or rotation requires pixel replication or more complex work (which often produces blocky results), and raster fonts may be stored in hardware ROM and less portable between systems.

Use stroke fonts when you need high-quality scalable text that participates in 3D transforms; use raster fonts when you need quick labels or HUD elements where speed matters and transformations aren't required.

Attributes (brief, concrete)

An **attribute** is a property of a geometric primitive that controls how it is drawn (color, thickness, fill pattern, edge display, etc.). Attributes are distinct from the primitive type itself — they change the appearance, not the topology.

Examples and common attribute controls:

- **Points**: color, size (point size).

- **Lines**: color, line width, stipple/pattern.
- **Polygons**: interior color/pattern, edge visibility, edge color/width, fill mode (solid/wireframe).

OpenGL calls that set attributes include (classic API examples):

- `glColor3f(r,g,b)` — sets current drawing color.
- `glClearColor(r,g,b,a)` — set clear color.
- `glLineWidth(width)` — set line thickness.
- `glPolygonMode(face, mode)` — control fill versus line rendering.

Attributes live in the OpenGL state machine; once set they affect subsequent primitives until changed again.

Lines and Polygons — practical reminders

- **Line primitives**: `GL_LINES`, `GL_LINE_STRIP`, `GL_LINE_LOOP`. They differ in how vertices are connected (pairs, sequential open strip, sequential closed loop).
- **Polygon primitives**: `GL_POLYGON`, `GL_TRIANGLES`, `GL_QUADS`, `GL_TRIANGLE_STRIP`, `GL_QUAD_STRIP`, `GL_TRIANGLE_FAN`. Each offers a different vertex packing that can reduce the number of vertices you must send for mesh-like geometry (e.g., triangle strips reuse vertices to form many triangles efficiently).
- Polygons must ideally be **simple** (no self-intersections), **convex** (angles < 180° where expected), and **flat** (all vertices lie in a single plane) to guarantee correct and predictable rendering with the simplest algorithms.

Final focused tips (one-liners to remember)

- Sphere = latitude/longitude grid → quad strips + pole fans.
- Convert degrees → radians: multiply by π/180.
- Smaller angular step → smoother sphere, more polygons.
- Stroke text = vector (transformable); raster text = bitmap (fast).
- Attributes change appearance, not shape — set with `glColor*`, `glLineWidth`, `glPolygonMode`, etc.

If you want, I can convert the sphere code in the slides into a short, clean C/GLUT function (immediate-mode) that you can paste and run — I'll keep it minimal and matched exactly to the approach above.

Here's a clear, structured explanation of the **Immediate-Mode Graphics and Color Concepts in OpenGL (Unit 2)** — balanced between short paragraphs and compact points, following the exact content you provided while keeping it easy to read and remember.

Immediate-Mode Graphics

In **immediate-mode graphics**, primitives are **not stored** in the graphics system's memory. Instead, as soon as a primitive (like a line, point, or polygon) is defined, it is **immediately processed and displayed** on the screen.

- The **current attribute values** (such as color, line thickness, fill pattern, etc.) are part of the **graphics state**.
- When a primitive is drawn, the system uses whatever attribute values are active at that moment.
- Once drawn, the primitive **is not remembered** by OpenGL — only its image remains on the display.
- If the display is cleared or the image is overwritten, the primitive is **lost** and must be reissued to appear again.

This was the traditional method in **classic OpenGL (pre-3.0)**, where drawing was done between `glBegin()` and `glEnd()`. Modern OpenGL instead uses **retained-mode graphics** through vertex buffers (VBOs) for efficiency, but the principle of immediate drawing remains an important foundation concept.

Color

Color is one of the **most important attributes** in computer graphics because it defines the visual appearance of every geometric primitive.

- A **visible color** is defined by its **wavelength spectrum** ranging approximately from **350 nm (violet)** to **780 nm (red)**.
- The **human visual system** does not sense the full wavelength distribution but instead reduces it to **three numerical responses** — one from each type of cone cell in the eye (red, green, and blue sensitive cones).
- These three responses are called **tristimulus values**, forming the foundation of the **three-color theory**.
- If two colors produce identical tristimulus values, they appear **visually identical** to humans, even if their physical spectra differ.

Tristimulus Values and RGB Representation

Any visible color can be represented as a combination of three primary colors — **Red (R)**, **Green (G)**, and **Blue (B)**.

$$C = rR + gG + bB$$

- **R, G, and B** are the *unit primaries*.
- **r, g, b** are the *relative intensities* of each primary color.
- These three values form the **tristimulus components** of the color.

OpenGL's color model is based on this **RGB additive color theory**.

Color Modes

OpenGL and general computer graphics use two fundamental color systems:

1. Additive Color Mode

- Used in **light-emitting devices** like monitors, CRTs, and projectors.
- **Primary colors:** Red, Green, Blue (RGB).
- Colors are produced by **adding light** of these primaries to a black background.
  - Red + Green → Yellow
  - Red + Blue → Magenta
  - Green + Blue → Cyan
  - All three → White

2. Subtractive Color Mode

- Used in **printing, painting, and photography** where a white surface reflects light.
- **Primary colors:** Cyan, Magenta, Yellow (CMY).
- These colors **subtract light** from white to produce the desired color (e.g., cyan ink absorbs red light).
- Common in printers and pigment-based color systems.

Comparison: Additive vs Subtractive

Feature	Additive Color	Subtractive Color
Used in	Monitors, CRTs	Printing, Painting
Primaries	Red, Green, Blue (RGB)	Cyan, Magenta, Yellow (CMY)
Method	Adds light to a black base	Subtracts color from white base
Result	Produces lighter colors as more light is added	Produces darker colors as more pigments are added

---

Color Cube Representation

Color can be visualized as a **3D cube**, called a **color solid**, in RGB space.

- **Vertices of the cube:**
  - **Black (0,0,0):** all primaries off
  - **Red (1,0,0)**, **Green (0,1,0)**, **Blue (0,0,1):** one primary fully on
  - **Cyan (0,1,1)**, **Magenta (1,0,1)**, **Yellow (1,1,0):** combinations of two primaries
  - **White (1,1,1):** all primaries fully on

- The **main diagonal** from black to white represents **grayscale values** — equal intensities of R, G, and B.

Color Models in OpenGL

There are **two main color models** used in OpenGL:

---

1. RGB Color Model

The **RGB (Red-Green-Blue)** model is an additive color model where each pixel stores separate intensity values for R, G, and B.

- Each pixel's color is represented in memory using **24 bits**:

$$8 \text{ bits (R)} + 8 \text{ bits (G)} + 8 \text{ bits (B)} = 24 \text{ bits per pixel}$$

- This allows $2^{24} = 16,777,216$ possible colors.
- In OpenGL, colors are specified using the function:

```
c

glColor3f(GLfloat R, GLfloat G, GLfloat B);
```

- The values of **R, G, and B** range from **0.0 (no intensity)** to **1.0 (full intensity)**.

Example:

```c
glColor3f(1.0, 0.0, 0.0);   // Sets current drawing color to red
```

Once set, this color remains active until changed again because **color is part of the graphics state**.

---

2. RGBA Color Model

The **RGBA** system extends RGB by adding an **Alpha (A)** component used for **transparency and blending**.

- **A = 0.0:** Fully transparent (invisible)
- **A = 1.0:** Fully opaque

Example:

```
c

glClearColor(1.0, 1.0, 1.0, 1.0);
```

This sets the background color to **opaque white**.

Alpha is crucial for creating effects like **fog, translucency, or overlapping transparent layers**.

---

3. Indexed Color Model

The **Indexed Color** system is used when the frame buffer has limited bit depth (few bits per pixel).

- Each pixel stores only an **index value** instead of full RGB values.
- The index refers to an entry in a **color lookup table (palette)** that contains the actual RGB values.

Example:

- If each pixel has **8 bits**, there are $2^8 = 256$ possible color indices.
- The lookup table (palette) maps each index to an RGB color.

Functions:

```
c


glIndexi(element);                  // Set color index
glutSetColor(int index, GLfloat r, GLfloat g, GLfloat b);  // Define palette colors
```

Advantages:

- Efficient for systems with low color depth.
- The programmer can modify the color table dynamically for palette effects.

Disadvantages:

- Only a limited number of colors (e.g., 256) can be displayed **simultaneously**.
- Not suitable for high-color or photo-realistic graphics.

Setting Color Attributes in OpenGL

Some common OpenGL commands to set visual attributes:

```
c


glClearColor(1.0, 1.0, 1.0, 1.0);   // Sets background color to white
glColor3f(0.0, 1.0, 0.0);           // Sets drawing color to green
glPointSize(2.0);                   // Sets point size to 2 pixels
```

These attributes define how subsequent primitives appear when rendered.

Summary

- **Immediate-mode**: draws primitives instantly, without storing them in memory.
- **Color**: defined by RGB tristimulus values (red, green, blue).

- **Additive (RGB)** adds light; **Subtractive (CMY)** removes light.
- **RGB Model**: full-color per pixel; **RGBA** adds transparency.
- **Indexed Model**: limited-color palette using color lookup tables.
- **OpenGL Commands** like `glColor3f()` and `glClearColor()` manage color attributes during rendering.

This concludes the full explanation of **Immediate-Mode Graphics and Color Concepts** in OpenGL (Unit 2).
Would you like me to follow up with a compact "exam-revision summary sheet" version for this topic (1–2 pages, bullet-style for quick memorization)?

Below is a clear, complete explanation of the material you pasted — keeping every point from the slides, structured so it's easy to follow and remember. I'll cover the **control functions**, **viewing (orthographic & 2-D)**, **matrix modes**, **viewports/aspect ratio**, and then walk through the **3D Sierpinski gasket program** line-by-line so nothing is missed.

Control functions (GLUT)

Control functions let your program talk to the window system, create windows, set display modes and callbacks, and enter the event loop. The slides list the common GLUT control functions; here's what each does and why it's used:

1. **`glutInit(int *argcp, char **argv)`**
   - Initializes GLUT and establishes interaction with the window system.
   - You normally call it early in **`main()`** and pass the command-line **`argc, argv`** so GLUT can parse its own options.
2. **`glutCreateWindow(char *title)`**
   - Opens an OpenGL window and gives it the provided title string (shown in the window bar).
3. **`glutInitDisplayMode()`**
   - Sets properties of the display buffer you want (color model, double vs single buffering, depth buffer, etc.).
   - Example flags: **`GLUT_SINGLE`** or **`GLUT_DOUBLE`**, **`GLUT_RGB`**, **`GLUT_DEPTH`**.
4. **`glutInitWindowSize()`**
   - Specifies the initial window width and height in pixels.
5. **`glutInitWindowPosition()`**
   - Specifies the initial top-left position of the window on the screen.
6. **`glutDisplayFunc()`**
   - Registers the *display callback* — the function GLUT calls whenever the window must be redrawn (first shown, uncovered, resized, etc.). Put all your rendering code inside this callback (or call it from here).

7. **`glutMainLoop()`**
    - Starts GLUT's event processing loop. The program now waits for and dispatches events (keyboard, mouse, window events). If idle it waits (similar conceptually to **`getch()`** for input-driven programs). This call does not return.

Viewing

Viewing controls how scene geometry is mapped to the final image. If you don't explicitly set a view, OpenGL uses an orthographic default. The slides break viewing into orthographic/2D and matrix mode details.

Orthographic view

- The simplest and default OpenGL projection is **orthographic (parallel) projection**.
- Conceptually it places the camera at an infinite distance so projection lines are perpendicular to the projection plane: a 3D point (x, y, z) maps to $(x, y, 0)$.
- The default viewing *volume* is a cube centered at the origin with side length 2 (so x,y,z in [−1,1]).
- You can explicitly define a right-parallelepiped viewing volume with:

```
c

void glOrtho(GLdouble left, GLdouble right,
             GLdouble bottom, GLdouble top,
             GLdouble near, GLdouble far);
```

  Only objects inside that volume are visible.

2-D viewing as a special case

- 2-D viewing is simply a 3-D view where you restrict attention to the plane $z = 0$.
- The utility **`gluOrtho2D(left, right, bottom, top)`** is equivalent to **`glOrtho(left, right, bottom, top, -1.0, 1.0)`** and is convenient for purely 2D programs because it sets a near/far appropriate for 2D.

Matrix modes (modelview vs projection)

The pipeline applies concatenated transformation matrices. The two key matrices are:

- **Model-view matrix** — combines modeling (object) transforms and the view (camera) transform.
- **Projection matrix** — defines the projection (orthographic or perspective) that maps camera-space to clip space.

These matrices are part of OpenGL state and start as identity. **`glMatrixMode()`** selects which matrix subsequent matrix calls affect:

- **`glMatrixMode(GL_PROJECTION);`** — make the projection matrix active (useful when calling **`glOrtho`**, **`gluPerspective`**, etc.).

- `glMatrixMode(GL_MODELVIEW);` — switch back to model-view before rendering or applying model transforms.

Common sequence for a 2D rectangle view (as in the slides):

```
c


glMatrixMode(GL_PROJECTION);
glLoadIdentity();
gluOrtho2D(0.0, 500.0, 0.0, 500.0);
glMatrixMode(GL_MODELVIEW);
```

This creates a viewing rectangle 0..500 in both x and y and then returns to model-view mode (important to avoid confusion later).

Aspect ratio and viewports

- **Aspect ratio** = width/height of the viewing rectangle. Maintaining correct projection vs viewport mapping preserves object shapes.
- **Viewport** is the rectangular area of the window where OpenGL draws (defaults to entire window). You set it with:

  ```
  c

  void glViewport(GLint x, GLint y, GLsizei w, GLsizei h);
  ```

  where `(x,y)` is lower-left corner in pixels and `w,h` are width and height. When a context is attached to a window, the viewport defaults to the window size; on resize you commonly reset the viewport to the new `w,h`.

Typical reshape handler adjusts both viewport and projection to preserve aspect and perform Zoom in/out.

3D Sierpinski Gasket program — full walkthrough

The program in the slides generates a 3D Sierpinski gasket by recursively subdividing the faces of a tetrahedron. Below I explain each portion so nothing is missed.

Data and globals

```
c


typedef float point[3];

point v[4] = {
  {0.0, 0.0, 10.0},      // v0
  {0.0, 10.0, -10.0},    // v1
  {-10.0,-10.0,-10.0},   // v2
  {10.0,-10.0,-10.0}     // v3
};
int n; // number of recursive divisions (entered by user)
```

- `v[]` defines the four vertices of the initial tetrahedron in 3D space.
- `n` is requested from the user at runtime and controls recursion depth.

`triangle()` — draw one triangle face

```
c


void triangle(point a, point b, point c) {
  glBegin(GL_POLYGON);
    glVertex3fv(a);
    glVertex3fv(b);
    glVertex3fv(c);
  glEnd();
}
```

- Given three 3D points (vertices), this emits a polygon (triangle here) using the current OpenGL attributes (color, polygon mode, etc.).

`divide_triangle()` — recursive subdivision

```
c


void divide_triangle(point a, point b, point c, int m) {
  point p1, p2, p3;
  int i;
  if (m > 0) {
    for (i=0; i<3; i++) p1[i] = (a[i] + b[i]) / 2;
    for (i=0; i<3; i++) p2[i] = (a[i] + c[i]) / 2;
    for (i=0; i<3; i++) p3[i] = (b[i] + c[i]) / 2;
    divide_triangle(a, p1, p2, m-1);
    divide_triangle(c, p2, p3, m-1);
    divide_triangle(b, p3, p1, m-1);
  } else {
    triangle(a, b, c);
  }
}
```

- If `m > 0`, compute midpoints of each edge: `p1` midpoint of `a-b`, `p2` midpoint of `a-c`, `p3` midpoint of `b-c`.
- Recursively subdivide the three outer triangles (notice the central small triangle is *omitted* — that's the Sierpinski step).
- If `m == 0`, draw the triangle (base case).

This is the 2D triangle subdivision step applied on every triangular face; for a tetrahedron face subdivision you apply this per face.

`tetrahedron()` — apply subdivision to all faces

```
c


void tetrahedron(int m) {
  glColor3f(1.0, 0.0, 0.0); // first face color
  divide_triangle(v[0], v[1], v[2], m);
```

```
  glColor3f(0.0, 1.0, 0.0); // second face color
  divide_triangle(v[3], v[2], v[1], m);

  glColor3f(0.0, 0.0, 1.0); // third face color
  divide_triangle(v[0], v[3], v[1], m);

  glColor3f(0.0, 0.0, 0.0); // fourth face color
  divide_triangle(v[0], v[2], v[3], m);
}
```

- The function colors each face differently for clarity. Note face vertex order affects face winding and visibility (anticlockwise vs clockwise determines front vs back if backface culling is enabled).
- Each face is subdivided `m` times.

`display()` — rendering callback

```
c


void display() {
  glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); // clear color + depth
  glLoadIdentity(); // reset model-view matrix
  tetrahedron(n);   // draw subdivided tetrahedron
  glFlush();        // ensure commands are executed
}
```

- **`glClear`** clears both color and depth buffers: depth clearing is crucial for correct hidden-surface removal in 3D.
- **`glLoadIdentity()`** resets the current matrix (model-view) to identity so drawing starts from a known transform.
- **`tetrahedron(n)`** draws the full subdivided gasket.
- **`glFlush()`** forces execution of queued GL commands (useful when single-buffered; if double-buffered you'd use **`glutSwapBuffers()`**).

`myReshape()` — window reshape handler (zoom & keep aspect)

```
c


void myReshape(int w, int h) {
  glViewport(0, 0, w, h);
  glMatrixMode(GL_PROJECTION);
  glLoadIdentity();
  if (w <= h)
    glOrtho(-12.0, 12.0, -12.0 * (GLfloat)h / (GLfloat)w,
            12.0 * (GLfloat)h / (GLfloat)w, -12.0, 12.0);
  else
    glOrtho(-12.0 * (GLfloat)w / (GLfloat)h,
            12.0 * (GLfloat)w / (GLfloat)h, -12.0, 12.0, -12.0, 12.0);
  glMatrixMode(GL_MODELVIEW);
  glutPostRedisplay();
}
```

- **`glViewport(0,0,w,h)`** ensures the viewport matches the new window size.
- Switch to **`GL_PROJECTION`** and **`glLoadIdentity()`** before setting **`glOrtho()`** — so you change the projection matrix only.

- The `if` preserves aspect ratio: when width ≤ height you keep X extents fixed and scale Y extents by `h/w`; otherwise scale X extents by `w/h`. This prevents stretching. The near/far for `glOrtho` are set to −12..12 to include the tetrahedron volume.
- After setting projection you switch back to `GL_MODELVIEW`.
- `glutPostRedisplay()` requests that GLUT call the display callback again with the updated projection.

`main()` — program startup and GLUT wiring

```
c


int main(int argc, char **argv) {
  printf(" No. of Divisions ? ");
  scanf("%d", &n);

  glutInit(&argc, argv);
  glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB | GLUT_DEPTH);
  glutInitWindowSize(500, 500);
  glutCreateWindow("3D Gasket");
  glClearColor(1.0, 1.0, 1.0, 1.0);

  glutReshapeFunc(myReshape);
  glutDisplayFunc(display);

  glEnable(GL_DEPTH_TEST); // enable hidden-surface removal
  glutMainLoop();
  return 0;
}
```

- Prompts the user for `n` (recursion depth).
- Initializes GLUT and requests a display mode with a depth buffer (`GLUT_DEPTH`) so OpenGL can do depth testing. The example uses `GLUT_SINGLE` (single buffering) and `GLUT_RGB`. In double-buffered programs you would use `GLUT_DOUBLE`.
- Creates a 500×500 window, sets clear color to white.
- Registers `myReshape` and `display` callbacks.
- **Enables depth testing** with `glEnable(GL_DEPTH_TEST)`: this activates Z-buffer based hidden-surface removal so closer fragments occlude farther ones.
- `glutMainLoop()` starts processing events and never returns.

Key behaviors emphasized by the code

- **Hidden-surface removal**: the program clears and uses the depth buffer (`GL_DEPTH_BUFFER_BIT`) plus `glEnable(GL_DEPTH_TEST)` to ensure faces closer to the camera hide those behind them. Without this you'd see overlapping triangles rendered in draw order only.
- **Aspect preservation & zoom**: `myReshape` adjusts `glOrtho` to preserve aspect ratio and ensures the object fills the window appropriately when resized.
- **Recursive Sierpinski construction**: `divide_triangle` omits the central small triangle at every recursion, producing the fractal pattern across each tetrahedron face. Applying this to all four faces produces the 3D gasket.

- **Color per face**: the code sets different colors for faces so the structure and orientation are visually clear.

Final compact reminders (so you don't miss anything)

- Call `glutInit()` before creating windows.
- Set display mode to include `GLUT_DEPTH` if you need hidden-surface removal, and enable depth testing with `glEnable(GL_DEPTH_TEST)`.
- Use `glMatrixMode(GL_PROJECTION)` + `glLoadIdentity()` before `glOrtho()` or `gluPerspective()`, then return to `GL_MODELVIEW`.
- On window resize: set `glViewport(0,0,w,h)` and adjust projection to preserve aspect.
- Recursive subdivision computes midpoints and recurses on outer triangles to create the Sierpinski pattern; draw poles/triangles only at recursion base case.
- Use `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)` each frame to clear both color and depth buffers.

If you want, I can:
• produce a cleaned-up single-file C/GLUT program based exactly on these slides (compilable), or
• produce a one-page cheat sheet that summarizes the callbacks, matrix sequence, and reshape math for quick revision. Which do you prefer?

Below is a focused, faithful explanation of **Input and Interaction** from Unit 2 — keeping very close to the content you supplied, preserving all points, and arranging them so they're easy to read and remember. I'll use short paragraphs for the main ideas and concise lists where a compact summary helps.

Objectives (what this section covers)

This unit introduces the main ideas needed to make graphics interactive: the kinds of input devices (physical vs logical), how input can be obtained (modes), the event-driven style of input, and how to program event-driven input (the slides point to GLUT as the utility used for this). These topics form the basis for interactive graphics: seeing something on screen, picking it, and manipulating it in response.

Project Sketchpad — the interaction paradigm

Ivan Sutherland's **Sketchpad (1963)** established the classic interactive loop that many graphics apps still follow. The cycle is simple and powerful:

1. The user sees an object on the display.
2. The user points to or "picks" that object with an input device (light pen, mouse, trackball, etc.).
3. The object changes (it moves, rotates, morphs).

4. Repeat.

This basic loop—display, point/pick, change—is the core of interactive computer graphics.

Graphical input — three ways to think about devices

When designing interactive graphics you can describe input devices from three complementary viewpoints:

- **Physical properties** — the actual hardware you hold or touch: mouse, keyboard, trackball, light pen, tablet, joystick, spaceball. These define how the user produces signals.
- **Logical properties** — what your program actually receives via the API: a 2D position, an object identifier (a pick), a numeric value, or a stroke (sequence of positions). The same logical input can come from different physical devices.
- **Modes** — how and when the program obtains input: either on explicit request (request mode) or asynchronously as events arrive (event mode).

Keeping these three views separate helps you design flexible interaction: the same program logic can accept different hardware by working with logical input types.

Physical devices (common examples)

Typical physical devices listed in the slides include:
mouse, trackball, light pen, data tablet, joystick, and space ball.
Each device has different ergonomics and signal behaviors (absolute vs incremental) which affects how you interpret its data.

Incremental (relative) vs absolute devices

Some devices report an absolute position directly to the OS (for example, some tablets). Others report **incremental** or relative motion — e.g., a mouse reports movements (deltas) produced by rolling a ball or an optical sensor. With incremental devices you must integrate those deltas over time to obtain an absolute cursor position. Practical consequences:

- Integration can accumulate error; absolute position is harder to compute precisely.
- Sensitivity may vary; small physical motions can produce large or small logical motion depending on device settings.
- Devices like trackballs or joysticks are naturally incremental; tablets and some pens provide absolute coordinates.

Logical devices — input abstracted from hardware

When your code reads input using high-level functions (e.g., `scanf`, `cin`), it typically does not care *which* physical device provided the value. In graphics, logical devices formalize this idea: the program works with **positions, picks, strokes, valuators,** and **choices**, independent of whether the source was mouse, file, or another program.

Older graphics APIs (GKS, PHIGS) defined logical input types such as:

- **Locator** — returns a position (e.g., mouse or pen coordinates).
- **Pick** — returns an object identifier (the result of picking).
- **Keyboard** — returns text input (strings).
- **Stroke** — returns an array of positions (a drawn stroke).
- **Valuator** — returns a floating-point value (e.g., slider).
- **Choice** — returns one of a set of discrete options.

These logical types let programs be written in device-independent ways.

Client–server model and X Window input

The X Window System introduced a **client–server** model widely used on workstations and networks: the OpenGL program is the *client* and the display/pointing device is managed by a *graphics server*. In this architecture the server handles the physical device and the framebuffer; the client receives input events and sends drawing commands to the server. This separation supports remote displays and networked graphics.

Input modes — request vs event

Input devices typically have a trigger (a mouse button, a key press/release). How programs receive that input depends on mode:

- **Request mode**: the program explicitly asks for input and the device supplies it only when triggered. A keyboard line-edit that accepts characters until Enter is a typical request-mode pattern: the input is provided when the user presses the trigger (Enter).
- **Event mode**: devices generate asynchronous events when triggered (mouse clicks, keydowns, etc.). Events are placed in a queue and the program processes them as they arrive. Event mode is the norm for modern interactive GUI and graphics programs because multiple devices can trigger at arbitrary times; the program reacts to the queued events.

Event mode supports responsive, concurrent interaction from mouse, keyboard, and other devices.

Event-driven input and GLUT (programming input)

The unit lists **event-driven input** and specifically mentions **programming event input with GLUT**. In that model:

- The toolkit (GLUT in many teaching examples) receives device triggers from the windowing system and places corresponding events into a dispatch mechanism.
- The program registers **callbacks** for particular event types (mouse events, keyboard events, reshape events, display callbacks).
- When an event occurs, the toolkit invokes the registered callback, passing the logical input (position, button, key, etc.) to the program. This matches the event-mode model and cleanly separates event handling code.

(You supplied the objective that GLUT is used for programming event input; the slides did not list GLUT callback names here, so I've kept this as a precise conceptual description rather than listing specific API calls.)

Final note — display lists on server

In a client–server environment the server can keep a **display list** (a stored sequence of drawing commands). That means the server can redisplay the scene without the client resending all primitives across the network each time. This reduces bandwidth and latency for redisplay operations in networked graphics setups.

If you want, I can now:
• turn these logical types and modes into a small GLUT event-handling template (mouse + keyboard callbacks) to show how events map to logical inputs, or
• make a one-page crib sheet pairing each physical device to the logical device it usually implements (e.g., mouse → locator/pick, tablet → locator/stroke).

Tell me which and I'll provide it without extra fluff.

Event Types & Callbacks — explained clearly

Interactive OpenGL programs are driven by *events*: things that happen outside the normal sequential flow (window resized, mouse clicked, key pressed, etc.). You handle those events by registering **callback** functions, which the toolkit (GLUT in these slides) invokes when the corresponding event occurs. Below I walk through the important event types, how callbacks work, and the related mechanisms you'll use in real programs.

Event types (what can occur)

Events fall into a few practical categories:

- **Window events** — resize, expose (window uncovered and needs redrawing), iconify (minimize).
- **Mouse events** — clicking a button, releasing it.
- **Motion events** — moving the mouse while a button is pressed (motion) or moving it with no buttons pressed (passive motion).

- **Keyboard events** — pressing or releasing keys (ASCII keys and "special" keys like arrows or function keys).
- **Idle** — no input events; used to run background work or animations when the event queue is empty.

Each event type carries a small set of logical data: e.g., mouse events include which button and the x,y position; keyboard gives you the key code and mouse position; reshape gives you new window width/height.

Callbacks — the programming interface for events

A **callback** is simply a function you supply that GLUT (or another toolkit) will call for a particular event. You register callbacks during initialization; examples of GLUT registration functions:

- `glutDisplayFunc(mydisplay)` — display refresh
- `glutMouseFunc(mymouse)` — mouse button presses/releases
- `glutReshapeFunc(myreshape)` — window resizes
- `glutKeyboardFunc(mykey)` — ASCII key presses
- `glutIdleFunc(myidle)` — called when no other events are pending
- `glutMotionFunc(...)` and `glutPassiveMotionFunc(...)` — mouse motion with/without buttons pressed

The toolkit's event loop (`glutMainLoop()`) repeatedly looks at the event queue and, for each event, calls the appropriate user-supplied callback. If no callback is registered for an event, the event is ignored.

The display callback and redisplay strategy

Every GLUT program must provide a **display callback**. GLUT calls it whenever the window needs updating: when it opens, when resized, when uncovered, or when your program asks for a redraw. You register it with `glutDisplayFunc(mydisplay)`.

A common issue is multiple triggers causing many immediate display calls in one loop iteration. GLUT provides `glutPostRedisplay()` to avoid redundant immediate calls: it *marks* that a redisplay is needed and GLUT will call your display callback once at the end of the current event processing pass if the flag is set. Use `glutPostRedisplay()` from other callbacks (mouse, keyboard, idle) to request a redraw cleanly.

In the display callback you typically:

1. Clear buffers (`glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)`),
2. Draw the scene, and
3. Swap buffers (if using double buffering) or `glFlush()` (single buffering).

Mouse and motion callbacks — coordinates and usage

Mouse callback signature in GLUT:

```c
void mymouse(GLint button, GLint state, GLint x, GLint y)
```

- **`button`**: which button (e.g., **`GLUT_LEFT_BUTTON`**).
- **`state`**: **`GLUT_DOWN`** or **`GLUT_UP`**.
- **`x,y`**: pixel coordinates where the event occurred (origin usually top-left of window).

Important coordinate detail: OpenGL's conventional world coordinates often assume origin at bottom-left, while many window systems deliver **`y`** with origin at top-left (because screens refresh top→bottom). To convert the callback **`y`** to OpenGL-style bottom-left origin:

```c
y = window_height - y;
```

Use that corrected **`y`** when mapping mouse positions into world coordinates.

You can terminate a program simply by checking for a right-button press:

```c
if (btn == GLUT_RIGHT_BUTTON && state == GLUT_DOWN) exit(0);
```

To react to clicks (e.g., draw a square at the click), you can implement **`drawSquare(x,y)`** and call it from **`mymouse`**. If you want continuous drawing while a button remains pressed, use **`glutMotionFunc(drawSquare)`**; to draw in response to pointer movement without pressing any buttons use **`glutPassiveMotionFunc()`**.

Keyboard and special keys

`glutKeyboardFunc(mykey)` gives you ASCII keys:

```c
void mykey(unsigned char key, int x, int y) { ... }
```

You can check for **`q`**/**`Q`** to quit, perform toggles, etc. For non-ASCII keys (arrows, function keys) GLUT provides **special key** callbacks and defines constants like **`GLUT_KEY_F1`**, **`GLUT_KEY_UP`**. You can also query modifier state (Shift, Ctrl, Alt) with **`glutGetModifiers()`** to implement combined actions (and emulate a 3-button mouse on 1- or 2-button mice).

Reshape callback — handling window size changes

`glutReshapeFunc(myreshape)` supplies the new width and height:

```c
void myreshape(int w, int h)
```

Inside reshape you should:

1. Set the new viewport: **`glViewport(0, 0, w, h)`**.
2. Switch to projection matrix: **`glMatrixMode(GL_PROJECTION); glLoadIdentity();`**
3. Recompute your projection (e.g., **`gluOrtho2D`** or **`glOrtho`**) in a way that preserves aspect ratio or chooses how world coordinates map to the window. The slides give standard code to preserve shapes: if **`w <= h`** keep X range fixed and scale Y by **`h/w`**, otherwise scale X by **`w/h`**.
4. Switch back to **`GL_MODELVIEW`** and optionally call **`glutPostRedisplay()`** (GLUT posts a redisplay automatically at the end of the reshape callback).

Reshape is the normal place to set projection-related state because it's called when the window is first created and whenever its size changes.

Picking (object selection) — how to identify clicked objects

Picking is the process of identifying which application object the user selected with a pointing device. In principle you have a screen position and can map it back to world geometry, but the rendering pipeline and 2D projection make this non-trivial.

Three common approaches:

- **Hit list / selection buffer** (most general): render in selection mode so geometry writes object IDs into a buffer when it falls into a small pick region. OpenGL provides a selection mode for this (**`glRenderMode(GL_SELECTION)`**) and helper functions: **`glSelectBuffer()`**, **`glInitNames()`**, **`glPushName(id)`**, **`glLoadName(id)`**, **`glPopName()`**. You push names (IDs) to identify objects while rendering; the selection mechanism records hits for primitives that fall into the pick region.
- **Feedback mode** (**`GL_FEEDBACK`**): records primitives instead of drawing so you can examine what would have been drawn in a region.
- **Rectangular maps** or simple spatial techniques: precompute a 2D map or bounding rectangles that make hit-testing cheap; adequate for many 2D or UI-style applications.

Selection and feedback are supported by OpenGL's **`glRenderMode(mode)`** where **`mode`** is **`GL_RENDER`** (normal drawing), **`GL_FEEDBACK`**, or **`GL_SELECTION`**.

Display lists — store and reuse geometry

A **display list** is a recorded sequence of GL commands you can call later. Think of it like compiling a chunk of drawing code for efficient reuse (especially useful in client-server or networked contexts so you don't resend repeated primitives).

Basic list flow:

```c

GLuint id = glGenLists(1);
glNewList(id, GL_COMPILE);
// OpenGL commands to record
glEndList();

// Later
glCallList(id);
```

Be aware: state changes inside a list can persist after the list executes. To avoid surprises use **`glPushAttrib()`/`glPopAttrib()`** and **`glPushMatrix()`/`glPopMatrix()`** inside lists to localize state changes.

Display lists are also handy for hierarchical modeling (compose chassis + wheels lists to build a car). They're less relevant for modern shader-based pipelines but still a useful concept for learning and some legacy contexts.

Animation and double buffering

When animating you must avoid showing partially drawn frames. Using **double buffering** prevents flicker and partial renders:

- Request double buffering: **`glutInitDisplayMode(GL_RGB | GLUT_DOUBLE)`**.
- In your display callback, render to the back buffer and at the end call **`glutSwapBuffers()`** to present the completed image atomically.

If you use single buffering you may see partially drawn frames and must **`glFlush()`** carefully.

For continuous animation, use the **idle callback** (**`glutIdleFunc(myidle)`**). The idle function runs when the event queue is empty; typical pattern:

```c

void myidle() {
  t += dt;           // update state
  glutPostRedisplay(); // request redraw at end of loop
}
```

Redraws are then handled in **`mydisplay()`** which uses current time/state **`t`** to render the correct frame.

Because GLUT callbacks have fixed signatures, you often store shared state in globals (e.g., global **`float t;`**) that callbacks read and update.

Toolkits, widgets, and menus

Toolkits provide higher-level UI building blocks (widgets): menus, sliders, dials, text inputs. GLUT offers a minimal widget set, including pop-up menus.

To create a simple GLUT menu:

1. Create the menu and supply a callback: **`menu_id = glutCreateMenu(mymenu);`**
2. Add entries: **`glutAddMenuEntry("Clear Screen", 1); glutAddMenuEntry("Exit", 2);`**

3. Attach the menu to a mouse button: `glutAttachMenu(GLUT_RIGHT_BUTTON);`
4. Implement `mymenu(int id)` to respond to selections (e.g., id==1 → `glClear()`, id==2 → `exit(0)`).

You can also build submenus with `glutAddSubMenu()` and change callbacks dynamically, use timers, and leverage portable fonts (`glutBitmapCharacter`, `glutStrokeCharacter`).

Practical tips & reminders

- Always convert mouse `y` from window-system coordinates (top-left origin) to your OpenGL world (often bottom-left origin).
- Use `glutPostRedisplay()` instead of immediate `display()` calls to avoid redundant redraws.
- For smooth animations use double buffering and `glutSwapBuffers()`.
- Preserve matrix and attribute state when calling display lists to avoid unexpected global effects.
- Use reshape callback to adjust viewport and projection consistently.
- Use selection/feedback modes or spatial data structures for reliable picking in 3D scenes.
- Keep shared state in globals or a small global struct since GLUT callbacks are fixed in signature.

If you want, I can now:

- produce a concise GLUT template showing all common callbacks (display, reshape, keyboard, mouse, motion, idle) wired together, or
- give a minimal, working C/GLUT example illustrating picking via the selection buffer, or
- prepare a one-page cheat-sheet for memorizing which callback arguments contain which information.

Tell me which one and I'll provide it exactly (no extra fluff).